

PROMIN STANDARD

promin

Зміст

Основний стандарт	4
Канонічні власники	4
Склад пакета	4
Призначення і межі	5
Семантична модель і гранулярність	5
Стійкі сутності	5
Відношення	5
Межа похідних даних	6
Інвентаризація	6
Ініціалізація	7
Замикання реалізації	8
Повноваження і контроль	8
Можливості	9
Розділення обов'язків	9
Завдання, оренда, докази та рішення	9
Матриця команд життєвого циклу	10
Прив'язка мутаційної команди до оренди	10
Evidence і незмінний CAS	11
Події, відтворення і цілісність	12
Проекція, пошук і продовження	13
Команди і профілі	14
Перевірка, міграція і перехід	15
Перевірка залежностей	16
Межі технологій і провайдерів	17
Робочі приклади	18
Кандидат стандарту, докази й рішення	19

Перевірка документів 20

Основний стандарт

Канонічна версія стандарту: 1.0.0-alpha.4. promin визначає мінімальний, перевірюваний стандарт керування агентною роботою, де семантика, повноваження, докази та події мають однозначних власників. JSON Core є нормативним джерелом; цей текст є згенерованою робочою проекцією тих самих фактів.

Власник: core/promin.manifest.json | Валідатор: bundle_digest + core_components

Канонічні власники

Core file	Sole responsibility
core/promin.manifest.json	Єдиний власник ідентичності Core bundle і точного digest binding п'яти інших Core artifacts.
core/semantic-model.json	Єдиний власник мінімального persistent vocabulary, relation grammar, межі derived projection і provider capability contracts.
core/authority-model.json	Own action capabilities, runtime Grant semantics, trust modes, configured signature-adapter verification, root ceilings, and separation-of-duty constraints.
core/policy-set.json	Єдиний власник executable cross-record invariants через стабільні validator identifiers без дублювання budget values.
core/conformance.json	Єдиний власник acceptance predicates, mutation families, structural hard ceilings, scale contracts і production-claim rules.
core/contracts.schema.json	Єдиний власник усіх structural, format, kind-specific payload і strict critical-field contracts в одному bundle.

Вибраний preset перебуває поза ідентичністю Core. Він обирає один обмежений operating profile і потреби провайдерів, але не надає повноважень. Схема є детермінованою проекцією Draft 2020-12, скопійованою з канонічних власників, а не другим власником семантики чи політики.

Власник: core/promin.manifest.json; presets/semantic-standard.json | Валідатор: exact six-file Core + selected preset outside Core

Склад пакета

Канонічне дерево містить рівно 226 звичайних файлів у 16 задекларованих директоріях. MANIFEST.json перелічує 224 payload-файлів; MANIFEST.json, SHA256SUMS.txt є згенерованими поверхнями цілісності. Відсутні, додаткові, linked, special або transient paths відхиляються.

Location	Regular files
package root	14
.github/	1
capability_profiles/	2
core/	6
docs/	25
examples/	1
human/	4
language_profiles/	7
presets/	1
profiles/	12
promin/	56
prompts/	2
skills/	4
tests/	76
tools/	15

Root files: рівно .gitattributes, .gitignore, CONTRIBUTING.md, LICENSE, MACHINE_README.md, MANIFEST.json, NOTICE, README.md, SECURITY.md, SHA256SUMS.txt, THIRD_PARTY_NOTICES.md, TRADEMARKS.md, VERSION.json, pyproject.toml. License, governance, dependency і third-party notice closure є частиною package identity.

Власник: MANIFEST.json; SHA256SUMS.txt; tools/promin_validate.py | Валідатор: CANONICAL_PACKAGE_FILES + CANONICAL_PACKAGE_DIRECTORIES

Призначення і межі

Стандарт відокремлює нормативний стан від похідних індексів і звітів, повноваження від ролей та оренди, а результат виконання від рішення про прийняття. Тип моделі може змінювати глибину аналізу, декомпозицію і паралельність, але не розширює повноваження.

- promin не є системою розподіленого консенсусу.
- Проекція, пошуковий рейтинг, звіт або preset не є нормативним джерелом і не надає повноважень.
- Оренда не є прийняттям; виконання, перевірка, просування і фінальне рішення є різними діями.
- Успіх еталонного пакета не є достатнім для production credit без закриття P0/P1, parity rollback/recutover і належно авторизованого рішення.

Власник: core/semantic-model.json; core/authority-model.json; core/conformance.json | Валідатор: model_tier_rule + production_claim_rule

Семантична модель і гранулярність

Семантична, публічна, compilation, runtime-dispatch і documentation гранулярності незалежні. Окремий тип або об'єкт виправданий лише незалежним інваріантом, політикою, замінністю, вимірюваним алгоритмом, межею відмови, повторним використанням або доменною значущістю.

Власник: core/semantic-model.json | Валідатор: design_rules

Стійкі сутності

Тип	Призначення
Завдання	Представляти одну плановану одиницю з одним acceptance predicate та обмеженою mutation score.
Candidate	Identify either one provider-proven immutable product snapshot or one explicitly non-creditable observation, independently from control state and bound to the exact candidate recipe.
Artifact	Identify immutable product, evidence, log, report, or diff bytes with provenance and retention metadata; raw-file inventory produces only derived Artifact proxies, while a diff may carry the canonical Candidate-delta change manifest without creating another persistent entity.
Run	Записувати одну спробу execution або validation щодо точних candidate, policy, tool та input digests.
Finding	Записувати спостережений дефект або порушення policy з явною effective-blocking semantics.
Decision	Record a project-operational acceptance, rejection, resolution, waiver, promotion, or release judgement. It is distinct from the external StandardReleaseDecision for the standard distribution.
GateResult	Record one gate outcome against the exact immutable Task-owned GateRunDefinition, current Run, and finalized evidence Artifact records; skipped, blocked, failed, stale, or unresolved evidence never receives pass credit.
Grant	Доводити одну capability subject у межах однієї Activation, time interval, nonce і scope.
Оренда	Координувати тимчасове володіння мутацією через generation, fencing, expiry, heartbeat і підтвердження закриття.

Власник: core/semantic-model.json | Валідатор: admission.entity

Відношення

Тип	Джерело	Ціль	Правила
DEPENDS_ON	Завдання	Завдання	acyclic-for-ready-frontier, accepted-current-outcome-required
READS	Task, Run	Candidate, Artifact	read-does-not-imply-authority
PRODUCES	Task, Run	Candidate, Artifact, Finding, Decision	content-or-record-identity-required
VALIDATES	Run	Candidate, Artifact, Task	candidate-policy-tool-input-binding-required
CITES	Finding, Decision	Artifact, Run, Finding	selector-and-content-resolution-required
BLOCKS	Finding	Task, Candidate, Decision	effective-only-while-open-and-blocking

Власник: core/semantic-model.json | Валідатор: admission.relation

Межа похідних даних

- InventoryEntry
- SearchDocument
- InventoryProjectionRow
- ProjectionLimits
- RetrievalPage
- ReadyFrontier
- WorkCardProjection
- WorkCard
- NextResult
- ContinueResult
- NavigationView
- StateCheckpoint
- CurrentReleaseStatus
- DoctorResult
- OperationMetrics
- Report

Each raw inventory file yields exactly one derived Artifact proxy. Raw-file proxies never create a persistent Task or relation; persistence still requires an independent invariant.

Власник: `core/semantic-model.json` | Валідатор: `derived_projection_kinds`

Інвентаризація

Інвентаризація читає дерево продукту один раз і записує незмінний JSONL stream, перевірений manifest. Кожний рядок нормалізується та входить до rolling digests ідентичності й stream, кількості bytes і рядків; публікація використовує atomic replace. Перебудова проєкції перевіряє descriptor і споживає stream один раз без матеріалізації corpus у пам'яті. Один прийнятий сирий файл створює рівно один похідний Artifact proxy з `data_class=untrusted-source`. Інвентаризація не синтезує Task, READS або PRODUCES.

InventoryProjectionRow	Структурне правило
digest	\$ref="#/\$defs/Digest"
path	\$ref="#/\$defs/RelativePath"
record_type	const="InventoryProjectionRow"
search_text	type="string"; minLength=1; maxLength=4096
semantic_proxy	type="object"; additionalProperties=false
size	type="integer"; minimum=0
Інваріант потоку	
Значення	
manifest_verified_jsonl	True
project_tree_passes_max	1
memory_amplification_max	32
derived_artifact_proxies_per_raw_file	1
search_text_bytes_max	4096

- P-008 minimal-graph: Each verified raw file creates exactly one non-authoritative derived Artifact proxy. Raw inventory creates no Task, READS, or PRODUCES record; Tasks and typed relations require explicit semantic content.

- P-017 single-scan-inventory: Один запит inventory виконує не більше одного повного проходу product tree; усі проєкції походять із його stream.
- P-043 candidate-snapshot-consistency: The candidate recipe is applied in the single inventory pass; promin v1 permits only provider-proven immutable VCS trees for credit, while best-effort observations are explicit and receive no pass credit.
- P-051 verified-inventory-provenance: Public rebuild accepts only a verified InventoryResult or its persisted manifest; caller provenance fields are reserved, data_class is forced to untrusted-source, and runtime injects the exact path and digest.

Власник: core/semantic-model.json; core/policy-set.json; core/contracts.schema.json | Валідатор: InventoryProjectionRow + manifest-verified JSONL + single_scan_inventory + inventory_projection_minimality

Ініціалізація

`promin init` приймає явно задані standard bundle, preset і рівно п'ять plan files: project.json, standards.json, technologies.json, licenses.json та authority.json. Team-signed trust додатково вимагає явний JSON array через --activation-proofs; local-owner сам утворює один root proof і відхиляє цю option. Init перевіряє Core, preset, доступність провайдерів і ліцензії до commit, встановлює незмінний стандарт за digest та атомарно записує рівно п'ять init records: ProjectInit, StandardsInit, TechnologiesInit, AuthorityInit і Activation. licenses.json є required validation input, а не шостим installed record. Успішні provider observations digest-bound у transient in-memory preflight receipt; цей receipt не є init record і ніколи не записується в `.promin/init/`. `--emit-plan` перевіряє та canonicalizes same supplied plans поза `.promin/`, не синтезує їх із project ID або source path і відхиляє --activation-proofs; `--review-plan` перевіряє inputs без запису; `--dry-run` додає provider preflight без мутації. Кожний режим init виконує нуль проходів дерева продукту. Product repository не потребує root core/; authority є лише exact immutable copy у `.promin/standard/<digest>/`.

Встановлений запис	Схема	Обов'язкові поля
project.json	ProjectInit	record_type, project_id, roots, candidate_recipe, preset_id, operating_profile, intent, resolved_profile, orchestration, telemetry
standards.json	StandardsInit	record_type, bindings
technologies.json	TechnologiesInit	record_type, bindings
authority.json	AuthorityInit	record_type, trust_mode, subjects, roots
activation.json	Activation	record_type, canonical_name, core_bundle_digest, preset_digest, init_file_digests, operating_profile, trust_mode, activation_digest, proofs, implementation_closure_digest

Власник: core/contracts.schema.json; core/policy-set.json | Валідатор: ProjectInit/StandardsInit/TechnologiesInit/AuthorityInit/Activation + exact_init_keyset

```
.promin/
  standard/<core-bundle-digest>/
    core/
      presets/<preset-digest>.json
  init/
    project.json
    standards.json
    technologies.json
    authority.json
    activation.json
  state/
    events/batches/
    objects/sha256/
    projection/
    head.json
    locks/
  generated/
  cache/
```

- P-002 activation-binding: Кожний нормативний запис прив'язує точну активну конфігурацію Core, preset, init і profile.
- P-023 exact-init-keyset: Activation прив'язує рівно project.json, standards.json, technologies.json і authority.json.

- P-028 exact-standard-files: Вхідні Core і вибраний preset відхиляють symlink та неочікувані shadow files.
- P-030 configuration-exact-idempotency: Повторна ініціалізація успішна лише тоді, коли запитаний canonical configuration digest точно дорівнює наявній Activation; інакше повертається конфлікт.
- P-037 provider-preflight-before-activation: Кожна обов'язкова provider identity і healthcheck успішно перевіряється до commit ініціалізації; оголошений provider не є доказом доступності.

Власник: core/policy-set.json | Валідатор: activation_binding + exact_init_keyset + exact_standard_files + config_exact_idempotency + provider_preflight

Замикання реалізації

Кожна technology binding називає точні компоненти promin runtime, інтерпретатора Python, jsonschema, SQLite та adapter для відповідної capability. Activation і залежні Evidence та проєкції прив'язуються до сукупного implementation_closure_digest. Runtime обчислює його перед використанням; drift робить залежний поточний стан недійсним, не переписуючи історію.

Прив'язка	Required content
TechnologiesInit.bindings[*].implementation_closure	runtime, interpreter, components, adapters, closure_digest
Activation.implementation_closure_digest	aggregate current implementation identity
EvidenceBinding.implementation_closure_digest	implementation used to produce evidence

- P-022 provider-substitution-binding: A provider identity or implementation-closure change invalidates the Activation and every dependent continuation, evidence Artifact, projection, replay result, and current release closure; it never silently changes authority.
- P-044 provider-adapter-dispatch: Every technologies.json binding resolves to the adapter actually invoked and carries a recomputed capability-specific implementation closure for the promin runtime, Python interpreter, jsonschema or SQLite component where used, and adapter. Unsupported or drifted tuples reject, and dependent evidence binds the exact implementation closure digest.

Власник: core/contracts.schema.json; core/policy-set.json | Валідатор: \$defs/TechnologiesInit + \$defs/Activation + \$defs/EvidenceBinding + implementation_closure_binding

Повноваження і контроль

authority.json configures root ceilings and Activation only. Root command proofs bind one command intent and may issue only the first authority.manage Grant. Every Grant proof binds claim_digest; later Grants cite an active authority.manage issuer Grant or team signatures verified by the exact signature adapter selected in technologies.json.

Потужність моделі змінює декомпозицію, контекст, паралельність і глибину аналізу, але ніколи не змінює повноваження.

Default - deny. Максимальна delegation depth - 8. Кожний Grant прив'язує subject, capability, conjunctive effect scope, nonce, not-before й expiry, revocation state, exact child signed claim, issuer claim або configured team-signature boundary та exact Activation. Revocation Decisions і Events розв'язуються як immutable records; довільний Decision ID не є доказом.

Власник: core/authority-model.json | Валідатор: bootstrap_rule + default + model_tier_rule

Можливості

Можливість	Призначення
standard.activate	Активувати одну конфігурацію репозиторію, прив'язану до digest.
authority.manage	Видавати або відкликати runtime Grants у межах налаштованого root ceiling.
task.plan	Створювати, ділити, впорядковувати або блокувати Tasks.
task.execute	Виконувати один Task у межах його WorkCard і чинної Lease.
lease.manage	Отримувати, підтверджувати heartbeat, закривати, завершувати строк або відкликати Lease.
evidence.publish	Публікувати evidence Artifacts, прив'язані до candidate.
validation.evaluate	Оцінювати gates без повноважень на promotion або release.
finding.record	Записувати Findings.
finding.resolve	Resolve one Finding only with evidence bound to that Finding's canonical digest.
finding.waive	Waive one Finding only through an expiring exception-policy Decision, Finding-bound evidence, and separation of duties.
candidate.promote	Просувати перевірений Candidate.
release.decide	Record an immutable historical release judgement bound to an exact acceptance-closure and provider-binding digest; current eligibility remains derived.
export.create	Створювати очищений зовнішній пакет.
migration.manage	Виконувати cutover, rollback і deterministic recutover.
projection.rebuild	Перебудовувати або перевіряти ненормативні проєкції.
projection.read	Read one bounded current projection or resume its continuation under the exact current Grant claim; it grants no mutation, validation, promotion, product acceptance, or standard distribution authority.

Власник: core/authority-model.json | Валідатор: capabilities

Розділення обов'язків

ID	Правило	Обмеження
SOD-001	lease-is-not-acceptance	{"forbid_lease_derived_capabilities": ["validation.evaluate", "finding.resolve", "finding.waive", "candidate.promote", "release.decide"], "scope_kind": "lease"}
SOD-002	executor-not-release-decider	{"forbid_same_subject_for_same_candidate": ["task.execute", "release.decide"], "scope_kind": "candidate"}
SOD-003	validator-not-self-promoter	{"forbid_same_subject_for_same_candidate": ["validation.evaluate", "candidate.promote"], "scope_kind": "candidate"}
SOD-004	finder-not-sole-waiver	{"forbid_same_subject_for_same_finding": ["finding.record", "finding.waive"], "scope_kind": "finding"}
SOD-005	evidence-producer-not-distribution-decider	{"forbid_same_key_capabilities": ["evidence.produce", "standard.distribute"], "forbid_same_subject_for_same_candidate": ["evidence.produce", "standard.distribute"], "scope_kind": "candidate-and-key"}

Standard distribution success never grants product acceptance. Live product credit separately requires a creditable immutable-vcs-tree Candidate, all P0/P1 findings closed with exact target-bound evidence, current project release closure eligibility, rollback and recutover equivalence, and an authorized human project Decision. product_acceptance_pass remains false for promin v1 distribution evidence.

Власник: core/authority-model.json; core/conformance.json | Валідатор: SOD-001..SOD-005 + signed standard-distribution decision

Завдання, оренда, докази та рішення

Task і Lease змінюють стан лише за policy-owned state machines. Mutation WorkCard потребує чинної Lease, а read-only WorkCard не повинен її отримувати. Lease окремо прив'язує manager і holder Grants, generation, monotonic fence, heartbeat, expiry і close acknowledgement, але ніколи не надає acceptance credit. ACTIVE і CLOSING, а також EXPIRED або REVOKED без записаного reconciliation продовжують займати capacity.

Завдання	Дозволений наступний стан
BLOCKED	READY, CANCELLED

Завдання	Дозволений наступний стан
CANCELLED	-
COMPLETED	-
LEASED	RUNNING, READY, BLOCKED, CANCELLED
PLANNED	READY, BLOCKED, CANCELLED
READY	LEASED, BLOCKED, CANCELLED
RUNNING	COMPLETED, BLOCKED, CANCELLED
Оренда	Дозволений наступний стан
ACTIVE	CLOSING, EXPIRED, REVOKED
CLOSED	-
CLOSING	CLOSED, EXPIRED, REVOKED
EXPIRED	-
REVOKED	-

```
{'closed_release': {'requires_field': 'close_ack', 'slot_reusable': True, 'state': 'CLOSED'}, '
terminal_reconciliation': {'fencing_token_binding': 'exact-lease-fencing-token', 'field': '
capacity_reconciliation', 'generation_binding': 'exact-lease-generation', 'manager_capability': '
lease.manage', 'required_fields': ['reconciled_by', 'grant_id', 'grant_claim_digest', 'reconciled_at', '
generation', 'fencing_token'], 'slot_reusable_after_record': True, 'states': ['EXPIRED', 'REVOKED']}, '
unreconciled_slot_reusable': False}
```

Матриця команд життєвого циклу

Команда	Визначення можливості	Область дії
task.record	task.plan	{"command_kind": "task.record", "id_field": "task_id", "kind": "task", "mode": "payload-id"}
task.transition	task.plan	{"command_kind": "task.transition", "id_field": "task_id", "kind": "task", "mode": "payload-id"}
artifact.record	{"diff": "task.execute", "evidence": "evidence.publish", "log": "task.execute", "product": "task.execute", "report": "projection.rebuild"}	{"command_kind": "artifact.record", "id_field": "artifact_id", "kind": "artifact", "mode": "payload-id"}
run.record	{"execution": "task.execute", "validation": "validation.evaluate"}	{"command_kind": "run.record", "mode": "payload-multi-id", "targets": [{"id_field": "task_id", "kind": "task"}, {"id_field": "candidate_digest", "kind": "candidate"}]}
gate.record	validation.evaluate	{"command_kind": "gate.record", "id_field": "candidate_digest", "kind": "candidate", "mode": "payload-id"}
finding.record	finding.record	{"command_kind": "finding.record", "id_field": "finding_id", "kind": "finding", "mode": "payload-id"}
decision.record	{"accept": "validation.evaluate", "promote": "candidate.promote", "reject": "validation.evaluate", "release": "release.decide", "resolve": "finding.resolve", "revoke": "authority.manage", "waive": "finding.waive"}	{"command_kind": "decision.record", "grant_target_type": "Grant", "id_field": "target_id", "kind_field": "target_type", "kind_map": {"Candidate": "candidate", "Finding": "finding", "Task": "task"}, "mode": "payload-dynamic-id-or-referenced-grant-scope"}
lease.record	lease.manage	{"command_kind": "lease.record", "id_field": "task_id", "kind": "task", "mode": "payload-id"}

Власник: core/authority-model.json; core/contracts.schema.json | Валідатор: command_capability_rules + command_effect_scope_rules + \$defs/GateResult

Прив'язка мутаційної команди до оренди

Команда, прив'язана до Lease, допустима лише тоді, коли authorization доводить точну capability команди, holder_authorization незалежно доводить task.execute, обидва точні твердження Grant прив'язують той самий subject, Activation і conjunctive effect scope, зазначена Lease є поточною й ACTIVE у момент issued_at, а task, generation, fence, candidate, context і digests WorkCard збігаються точно.

Grant авторизації команди та holder Grant визначаються й перевіряються незалежно; вони можуть бути одним записом лише тоді, коли обов'язкова capability команди є task.execute.

Доказ авторизації	Джерело / поле	Обов'язкова можливість
Grant авторизації команди	CommandRequest.authorization.grant_id + grant_claim_digest	визначена capability команди
Grant утримувача	holder_authorization.grant_id	task.execute
Рівність прив'язки		Обов'язковий результат
CommandRequest.workcard_task_id == WorkCard.task_id == Lease.task_id		точний збіг
CommandRequest.subject_id == Lease.holder_subject_id		точний збіг
CommandRequest.holder_authorization.grant_id == WorkCard.holder_grant_id == Lease.holder_grant_id		точний збіг
CommandRequest.lease_id == WorkCard.lease_id == Lease.lease_id		точний збіг
CommandRequest.lease_generation == WorkCard.lease_generation == Lease.generation		точний збіг
CommandRequest.fencing_token == WorkCard.fencing_token == Lease.fencing_token		точний збіг
CommandRequest.context_digest == WorkCard.context_digest		точний збіг
CommandRequest.workcard_digest == sha256(canonical WorkCard)		точний збіг

Команди, прив'язані до оренди: artifact.record, run.record, gate.record, finding.record.
Обов'язковий стан Lease: ACTIVE. Stale або unresolved: reject.

Власник: core/authority-model.json; core/policy-set.json | Валідатор: command_mutation_claim_rule + lease_fence + command_capability_scope

Evidence і незмінний CAS

Artifact ідентифікує незмінні bytes за digest у content-addressed storage. Credit завжди розв'язує точні artifact_id і digest фіналізованого Artifact record. Для artifact_kind=evidence event payload містить replay-complete metadata: точну Activation, Candidate, policy, tool, provider й input digests, outcome, evidence_purpose, evidence_class, stale, unresolved та product_credit_eligible. Клас harness-generated не змішується з product-execution і не отримує product credit.

Поле evidence	Правило
digest	незмінна ідентичність вмісту CAS
evidence_binding	activation_digest, implementation_closure_digest, candidate_digest, policy_digest, tool_digest, input_digests
outcome	pass, fail, blocked, skipped, error
evidence_class	product-execution, validator, harness-generated, migration
evidence_purpose	must equal the precommitted GateRunDefinition purpose
stale / unresolved	обидва false для product credit
product_credit_eligible	true лише для свіжого, вирішеного, успішного evidence класу product-execution

Gate statuses pass, fail і blocked прив'язують точний evidence до незмінного GateRunDefinition, заздалегідь визначеного його Task-власником. Skipped потребує reason, не має Artifact credit і завжди має pass_credit=false. Definition фіксує клас і purpose evidence, вимогу product credit, точні target kind, digest і scope, а також Candidate, policy, tool, provider та input digests до подання Run або GateResult. Inline definitions, змішані класи й заміна digest payload відхиляються.

Власник: core/contracts.schema.json; core/semantic-model.json; core/policy-set.json | Валідатор: \$defs/Artifact + \$defs/EvidenceBinding + \$defs/GateResult + candidate_evidence_binding + credit_requires_evidence

- P-004 lease-fence: Кожна мутація використовує поточну активну generation Lease та монотонний fencing token.
- P-005 candidate-evidence-binding: Every evidence Artifact requires an explicit diagnostic, gate, or product purpose and binds Activation, candidate, policy, provider/tool, and input digests inside the immutable event payload.
- P-013 safe-export: Export використовує allowlist, рекурсивно обмежений, безпечний щодо symlink і secrets та очищує paths.

- P-014 closed-acknowledgement: Лише чинний перехід до CLOSED з явним підтвердженням закриття звільняє слот мутації.
- P-033 lease-only-for-mutation: Mutation WorkCards потребують поточної Lease; read-only WorkCards не повинні отримувати або нести mutation Lease.
- P-038 canonical-state-machines: Переходи Task і Lease приймаються лише state machines, власником яких є policy; generated lifecycle views не можуть вигадувати переходи.
- P-039 decision-target-binding: Кожний Decision називає одну типізовану ціль, а decision kind, requested scope, candidate і authority узгоджені з цією ціллю.
- P-040 credit-requires-evidence: Pass, fail, and blocked gate results cite at least one exact immutable Artifact record reference; skipped results never receive pass credit.

Власник: core/policy-set.json | Валідатор: lease_fence + candidate_evidence_binding + current_outcome + effective_finding + lease_only_for_mutation + state_machine_transition + decision_target_binding + credit_requires_evidence

Події, відтворення і цілісність

обмежений канонічний розбір
 -> скомпільована схема Draft 2020-12
 -> реєстр семантичних політик
 -> цілісність Activation
 -> визначення capability і conjunctive effect scope
 -> перевірки Grant, SOD, Lease і fence
 -> прив'язка Candidate та Evidence
 -> обмежений атомарний пакет подій
 -> відновлювана проєкція

Одна нормалізована команда з exact authorization, idempotency key та expected HEAD утворює один незмінний атомарний batch. Під cross-process writer lock runtime оновлює exact HEAD і replay-derived authority state, перевіряє встановлені bytes, авторизує та застосовує команду до копії, обчислює typed state delta, виконує file sync, atomic replace і directory sync та лише після durable append публікує live state. Batch містить рівно одну primary event, що відповідає validated command payload; event IDs унікальні. Commit має $O(\text{batch})$, пошук HEAD - $O(1)$, replay - $O(\text{events})$.

EventBatch commitment	Required meaning
previous_authority_commitment	exact previous commitment or the Core-owned genesis value
authority_commitment	commitment over sequence, previous batch, event and state-binding identities
cumulative_event_count	previous count plus current Event count
event_semantic_digest	ordered append-only digest of canonical Events
state_binding_delta	non-empty sorted unique set/delete changes to typed authority leaves
cumulative_state_binding_update_count	previous update count plus current delta length
state_binding_digest	typed-sparse-merkle-v1 post-batch authority root
created_at	UTC with exact second precision

Replay перевіряє повний префікс journal. EventStorePolicy є derived live-only з установлених owners без defaults або nullable critical fields. Checkpoints, SQLite, typed graph state і sidecars можуть прискорювати rebuild, але не можуть підставляти відсутній commitment, додавати authority leaf або змінювати значення journal.

- P-006 atomic-event-commit: Одна команда створює один immutable batch через lock, compare-head, sync, atomic replace і recovery.
- P-011 explicit-external-bindings: Зовнішні стандарти та providers існують лише в explicit init records, прив'язаних до digest.
- P-015 fail-closed-critical-fields: Відсутні, невідомі, stale, downgraded або unbound critical fields спричиняють відхилення до мутації.
- P-025 one-command-one-batch: Кожний committed batch прив'язує рівно один subject, validated command digest, command intent digest, authorization digest та idempotency key.
- P-027 strict-time-order: Timestamps використовують canonical UTC seconds; порядок issued, heartbeat, expiry, finish і decision перевіряється семантично.

- P-041 command-event-correspondence: Один command batch прив'язує точне authorization claim і містить рівно одну primary event, рівну validated command payload; auxiliary events є лише типізованими відношеннями, а event IDs унікальні.

Власник: core/authority-model.json; core/policy-set.json; core/conformance.json | Валідатор: EventStorePolicy + EventBatch commitments + atomic_event_commit + historical_revocation + state_command_idempotency + one_command_one_batch + strict_time + command_event_correspondence

Проекція, пошук і продовження

SQLite/typed graph є одною disposable projection. `next` не виконує FTS: він обчислює live-only ReadyFrontier з поточного ациклічного графа DEPENDS_ON, вибирає перший Task у детермінованому порядку frontier і повертає immutable read-only WorkCard у NextResult.work_card. `continue` споживає opaque ReadyFrontier token і повертає ContinueResult. Обидва result envelopes містять рівно шість полів і жодних інших: record_type, status, subject_id, activation_digest, work_card і continuation, саме в цьому contract order. status=ready вимагає strict Core WorkCard, status=empty вимагає work_card=null, а continuation є non-null лише для truncated frontier page і містить лише next-page token. Task є eligible лише тоді, коли всі dependencies мають поточний COMPLETED, усі потрібні GateResults є current pass з credit, а жодний поточний OPEN Finding не блокує Task. WorkCardProjection є лише internal selected-Task materialization, не є public result envelope і не містить continuation field або token.

Для інших generic bounded retrieval operations точний пошук ID передує content-aware FTS; RetrievalPage є окремим generic retrieval result, а ranking не надає фактів прийняття. Широкий запит вибирає лише детермінований bounded top-k seed set. Якщо збігів більше, RetrievalPage встановлює refinement_required=true, повертає bounded hints і не відкриває pagination через невибраний corpus. Typed closure обчислюється лише для вибраних seeds, а об'єднання вибраного closure є повним у межах depth 1-12 і budgets. Неявне скорочення заборонено.

Public scheduling continuation є opaque authenticated ReadyFrontier next-page handle розміром не більше 256 bytes. Його bound state зберігається локально, обмежений 16 KiB і містить subject, чинний Grant projection.read, scope, Activation, current ReadyFrontier, детермінований порядок і cursor, HEAD, projection, повні budgets, часові межі, стан revocation й implementation closure. NextResult.continuation і ContinueResult.continuation є null, коли frontier page завершена. Окремо RetrievalPage володіє generic retrieval continuation і містить лише stream_cursor та next_stream_cursor; видалені cursor aliases відхиляються. Його token не є ReadyFrontier scheduling continuation. WorkCardProjection не містить continuation field або token. Overhead continuation не перевищує десяти відсотків bounded WorkCard. Кожна сторінка повторно перевіряє Grant; token не надає повноважень і не може переходити невибраними збігами corpus.

Граничні значення WorkCard		Значення
max_bytes		16384
max_entities		32
max_fanout_per_entity		8
max_relations		48
top_k		12
Поле продовження	Структурне правило	
activation_digest	{"\$ref": "#/\$defs/Digest"}	
budgets	{"\$ref": "#/\$defs/Budget"}	
capability_id	{"const": "projection.read"}	
cursor	{"minimum": 0, "type": "integer"}	
dependency_depth	{"maximum": 12, "minimum": 1, "type": "integer"}	
expires_at	{"\$ref": "#/\$defs/Timestamp"}	
grant_claim_digest	{"\$ref": "#/\$defs/Digest"}	
grant_id	{"\$ref": "#/\$defs/Id"}	
head_digest	{"oneOf": [{"\$ref": "#/\$defs/Digest"}, {"type": "null"}]}	
head_event_id	{"oneOf": [{"\$ref": "#/\$defs/Id"}, {"type": "null"}]}	
head_sequence	{"minimum": 0, "type": "integer"}	

Поле продовження	Структурне правило
implementation_closure_digest	{"\$ref": "#/\$defs/Digest"}
issued_at	{"\$ref": "#/\$defs/Timestamp"}
normalized_query_digest	{"\$ref": "#/\$defs/Digest"}
projection_digest	{"\$ref": "#/\$defs/Digest"}
ranking_id	{"const": "bm25-v1"}
record_type	{"const": "ContinuationTokenClaims"}
requested_scope	{"items": {"\$ref": "#/\$defs/ScopeSelector"}, "maxItems": 32, "minItems": 1, "type": "array"}
revocation_epoch	{"\$ref": "#/\$defs/Digest"}
state_id	{"\$ref": "#/\$defs/Id"}
subject_id	{"\$ref": "#/\$defs/Id"}
token_version	{"const": 2, "type": "integer"}
traversal_id	{"const": "typed-bfs-v2"}
ttl_seconds	{"maximum": 3600, "minimum": 1, "type": "integer"}
Правило обмеженого пошуку	Значення
broad_query_behavior	deterministic bounded top-k seeds, refinement_required=true, no corpus pagination
continuation_token_bytes_max	256
continuation_state_bytes_max	16384
selected_closure_union_completeness	1
corpus_independent_output_bound	True

- P-007 projection-is-derived: Видалення проєкції та deterministic replay не можуть змінити нормативне значення.
- P-009 bounded-workcard: Every WorkCard stays within both Core hard ceilings and its selected profile byte, entity, relation, fanout, and top-k budgets; every truncation is explicit and losslessly resumable.
- P-024 stale-projection-rejection: Генерація status, next і WorkCard відхиляє проєкцію, source HEAD або Activation якої відрізняється.
- P-026 deterministic-retrieval: Однакові ranking scores розв'язуються детерміновано, а acceptance-critical facts додаються через typed closure, не fuzzy ranking.
- P-032 depth-aware-retrieval-budget: Effective retrieval seed count reserves entity and relation capacity for typed closure at the requested dependency depth; top_k is a ceiling, byte/entity/relation/fanout budgets all remain effective, and no removed or unknown budget field may bypass a ceiling.
- P-042 continuation-authorization-and-completeness: A continuation is authenticated by a random per-Activation secret and binds the exact subject, projection.read Grant claim, capability, requested scope, revocation state, normalized query, deterministic cursor, Activation, HEAD, projection, ranking, complete budgets, depth, implementation closure, issue time, and expiry; its complete page union emits every reachable entity and relation exactly once without loss.

Власник: core/policy-set.json; core/conformance.json; core/contracts.schema.json | Валідатор: ReadyFrontier + NextResult + ContinueResult + WorkCard + WorkCardProjection + RetrievalPage + projection_derived + bounded_workcard + stale_projection + deterministic_retrieval + depth_aware_retrieval + \$defs/ContinuationTokenClaims

Команди і профілі

Поверхня команд	Команди	
public	init, doctor, status, next, validate, static-admission, continue, audit, refresh, context, skills	
Команда	Arguments	Grant capability
next	--subject SUBJECT --grant HOLDER_GRANT --query-grant QUERY_GRANT [--depth 1..12]	holder=task.execute; query=projection.read
continue	TOKEN --subject SUBJECT --grant QUERY_GRANT	query=projection.read

Два Grants для next перевіряються незалежно. Next обчислює поточний ReadyFrontier і повертає його перший eligible Task як NextResult.work_card; він не приймає FTS expression. Continue потребує явного subject і чинного query Grant з projection.read та повертає ContinueResult для наступної frontier page. Обидва exact envelopes містять лише record_type, status, subject_id, activation_digest, work_card і continuation.

Профіль	Рівень	Паралельність	Типова глибина	Байти	Сутності	Відношення	top_k
baseline	weak-local	1	2	8192	16	24	8
balanced	capable	3	4	12288	24	36	10
extended	strong	6	6	16384	32	48	12

- full-scan-on-init
- provider-discovery-as-authority
- implicit-action-grant
- unbounded-context-expansion
- search-ranking-as-acceptance-proof
- P-052 supported-command-surface: The v1 preset advertises exactly init, doctor, status, next, validate, and continue; no optional or administrative command catalogue may claim an absent executable path.

Власник: presets/semantic-standard.json; core/authority-model.json | Валідатор: base_user_commands + profiles + forbidden_implicit_behavior + model_tier_rule

Перевірка, міграція і перехід

Однаковий суворий pipeline застосовується до command, import, replay, rebuild і export. Порожні, невідомі, null, stale, degraded, unbound або cyclic critical records відхиляються. Одноразовий інструмент поза canonical promin імпортує підтримувані Tasks, Grants і revocations, Leases, evidence Artifacts, Findings, GateResults, Decisions та continuation state через той самий ingress; generated indexes, reports і DB перебудовуються. Cutover потребує candidate-bound dual-run parity, rollback, deterministic recutover, закриття P0/P1, нуль legacy references і належно авторизованого людського Decision. До цього cutover=false і deletion=false.

ResearchDraftIntake є bounded transient ingress. Кожне джерело прив'язується до immutable Artifact receipt, recomputed content digest, size, media type, provenance й active reviewed або source-verified license plan. Claim kinds: research_question, hypothesis, assumption, evidence_claim і blocked_claim; verification: unverified, source-bound, verified або blocked. Citation refs ідентифікують research sources, а evidence refs - незалежно resolved Artifact records. Metadata-only sources залишають усі залежні claims blocked і normative_use_allowed=false.

SanitizedResearchDraft містить лише eligible source-bound або independently verified factual candidates. Distribution, marketing і proof claims залишаються в excluded ledger з bounded text і digest, source/evidence refs та reasons. Result є derived live-only і не може змінювати GateResult, Decision, acceptance, pass credit, public approval або distribution state; public_release_approved залишається false.

SemanticExport є export-only semantic-state record, не package і не authority. Він прив'язує exact Candidate, Activation, HEAD, Core, preset, implementation closure, provider receipt, inputs та фактичні output Artifact ID і finalized record digest. Output bytes, digest, size і media type повторно обчислюються з exact CAS bytes; exported records deduplicate за digest і впорядковуються digest-ascending. Rebuild перевіряє journal та inventory inputs, виконує нуль product passes і атомарно публікує disposable projection.

canonical-name-only, exact-six-core-artifacts, preset-outside-core-digest, exact-one-selected-preset, exact-five-init-records, zero-product-scan-during-init, single-pass-inventory, no-false-success, strict-critical-fields, activation-digest-binding, root-ceiling-without-bootstrap-grant-cycle, grant-scope-expiry-nonce-proof, lease-generation-fence-and-close-ack

candidate-evidence-provider-binding, one-command-one-atomic-batch, deterministic-replay, projection-disposable-and-current, bounded-workcard-through-depth-twelve, typed-reference-domain-range-closure, dependency-graph-acyclic-and-current-ready-frontier, safe-recursive-export, idempotent-state-commands,

normalization-collision-rejection, provider-substitution-invalidates-derived-state, rollback-and-recutover-equivalence, zero-forbidden-name-occurrences

zero-unowned-normative-facts, zero-core-provider-lock-in, zero-compiled-cache-files, physical-100k-one-artifact-proxy-per-file, external-standard-release-decision-exact-binding, root-bootstrap-command-limited, deterministic-activation-identity, configuration-exact-idempotency, truthful-operation-reporting, depth-aware-seed-budget, lossless-continuation-in-reference-100k-scenario, read-workcard-without-lease, mutation-workcard-requires-current-lease

command-capability-and-scope-resolution, grant-issuance-chain-closure, team-signatures-cryptographically-verified, required-provider-preflight-before-commit, policy-owned-task-and-lease-state-machines, decision-kind-target-scope-binding, non-skipped-credit-result-has-evidence, exact-command-primary-event-correspondence, conjunctive-scope-containment, authorization-binds-exact-grant-claim, command-effect-target-in-requested-scope, lease-holder-has-task-execute-grant, decision-grant-matches-command-authorization

artifact-kind-resolves-correct-capability, continuation-v2-complete-page-union, candidate-recipe-applied-once, creditable-candidate-immutable-vcs-tree, selected-provider-adapter-evidence, team-signed-cli-end-to-end, current-release-closure-revalidation, finding-bound-resolution-and-waiver-evidence, candidate-delta-within-task-and-grant-scope, verified-head-checkpoint-delta-replay, profile-default-with-core-depth-twelve, canonical-exact-zip-distribution, current-interpreter-and-clean-install-modes

required-no-degradation-tests-fail-closed, continuation-secret-subject-grant-binding, implementation-closure-bound-and-current, verified-inventory-result-provenance, exact-six-base-command-surface, zero-dead-evidence-budget-fields, profile-bound-physical-100k-performance, windows-linux-exact-zip-matrix, fresh-semantic-control-state-per-saturation-iteration, three-zero-new-saturation-iterations, standard-distribution-separate-from-product-acceptance, candidate-evidence-decision-acyclic-chain, evidence-manifest-physical-resolution

evidence-manifest-exact-eight-lane-matrix-resolution, evidence-manifest-four-cp314-supplemental-records, evidence-manifest-path-digest-size-closure, evidence-manifest-nested-and-supplemental-chronology, seven-distinct-role-producer-scopes, signed-standard-decision-authority-proof, semver-from-candidate-binding, typed-human-document-verification, broad-query-bounded-seed-refinement, continuation-token-bytes-at-most-256, inventory-stream-memory-amplification-at-most-32, explicit-scale-marker-selection, authenticated-role-platform-evidence-attestations

installed-platform-observation-closure, handle-relative-evidence-root-safety, raw-scale-summary-recomputation, bounded-incremental-commit-and-compaction, offline-installed-no-degradation, offline-release-online-compatibility, decision-after-evidence-with-bounded-skew, single-derived-platform-closure-owner, truthful-process-invocation-status, product-public-approval-separation, historical-decision-current-eligibility-separation

Власник: core/policy-set.json; core/conformance.json; core/contracts.schema.json | Валідатор: ResearchDraftIntake + SanitizedResearchDraft + SemanticExport + SemanticStatePayload + ReadyFrontier + required_acceptance + mutation_families + Draft 2020-12 oneOf

Перевірка залежностей

Режим	Фактичне виконання
current-environment	check the interpreter running the command; do not create a nested environment
offline-wheelhouse	create a clean temporary environment; install only from the explicit wheelhouse
online-clean	create a clean temporary environment; install declared dependencies online
none	omit only the dependency smoke and grant no pass credit

No-degradation виконує кожний обов'язковий тест і відхиляє пропущений обов'язковий тест. Він cross-bind контролерну platform, версію CPython, ABI і tags з executable та SHA-256 чистого venv, base executable у межах base prefix та його SHA-256, що збігається з контролером, і одну версію SQLite у controller, validation та installed observations. Один monotonic total deadline охоплює setup, копіювання, створення середовища, встановлення залежностей, probes, validation і tests. Кожний subprocess ізольований як process tree, а залишкові descendants завершуються після timeout або виходу parent. Вичерпання deadline, output overflow, orphaned descendants або відсутність progress відхиляє результат без pass credit.

Рядки матриці platform і Python є лише audit-level спостереженнями сумісності. Вони не є нормативними, не надають standalone pass credit і не можуть авторизувати дистрибуцію або product acceptance. Додаткові рядки інтерпретаторів не додають ролей EvidenceManifest.

Кожна saturation iteration використовує свіжий exact п'яти-record, zero-scan control state і ніколи не повторно використовує semantic state. Вхідний baseline, control state кожної iteration і відновлений baseline зберігають byte identities. Audit output містить лише evidence та logs; recoverable control state зберігається в окремому operational-state root поруч із workspace. Partial progress не отримує credit, а фінальний evidence публікується лише після трьох послідовних повних zero-new iterations.

Режим масштабування	Фактичне виконання
python -B -m pytest -q -m "not scale"	focused режим відповідності; фізична scale lane не виконується

Режим масштабування	Фактичне виконання
PowerShell: \$env:PROMIN_SCALE_WORKSPACE='C:\promin-scale'; \$env:PROMIN_SCALE_ARCHIVE='C:\evidence\promin.zip'; python -B -m pytest -q -m scale	Windows physical lane на 100000 файлів; workspace і точний архів задано явно
POSIX: PROMIN_SCALE_WORKSPACE=/tmp/promin-scale PROMIN_SCALE_ARCHIVE=/tmp/evidence/promin.zip python -B -m pytest -q -m scale	Linux physical lane на 100000 файлів; workspace і точний архів задано явно

Власник: tools/promin_validate.py; tools/promin_package.py; tools/promin_no_degradation.py; tools/promin_saturation_audit.py |
Валідатор: dependency-mode dispatch + runtime cross-binding + total deadline + process-tree containment + fresh saturation control state

Межі технологій і провайдерів

Можливість	Завдання	Заборонена межа	
control-runtime	Розбирати explicit configuration, канонізувати JSON і виконувати local CLI orchestration.	{"effect": "reject", "forbids": ["authority-invention", "policy-invention", "provider-binding-invention"], "normative_authority": false}	
shape-validation	Перевіряти єдиний contract bundle Draft 2020-12 з обов'язковими formats і local references.	{"effect": "reject", "forbids": ["authority-decision", "cross-record-semantic-decision"], "normative_authority": false}	
content-identity	Обчислювати content digests та ідентичності content-addressed objects.	{"effect": "reject", "forbids": ["digest-as-action-authority"], "normative_authority": false}	
local-serialization	Надавати один local-writer lock, atomic replace, file sync, directory sync і recovery primitives.	{"effect": "reject", "forbids": ["distributed-consensus-claim"], "normative_authority": false}	
query-projection	Build disposable status and bounded typed-retrieval projections; derive ReadyFrontier, materialize one exact WorkCardProjection, and return Core-owned NextResult or ContinueResult envelopes for the six supported CLI commands.	{"effect": "reject", "forbids": ["projection-as-normative-authority"], "normative_authority": false}	
filesystem-inventory	Apply the exact candidate recipe in one product pass, use no-follow reads, and publish either a provider-proven immutable VCS tree stream or an explicit non-credible observation.	{"effect": "reject", "forbids": ["implicit-initialization-scan"], "normative_authority": false}	
export-scan	Виконувати bounded recursive export inspection, allowlisting, secret classification і відхилення symlink.	{"effect": "reject", "forbids": ["package-as-authorization-boundary"], "normative_authority": false}	
signature	Dispatch the exact configured signature adapter to sign or verify activation, Grant, and Decision digests for team trust mode, and evidence the invoked adapter identity.	{"effect": "reject", "forbids": ["core-signer-implementation-lock-in"], "normative_authority": false}	
build-dependency	Імпортувати project-specific build і dependency facts як похідні проєкції.	{"effect": "reject", "forbids": ["semantic-model-redefinition"], "normative_authority": false}	
Клас провайдера preset		Ідентифікатори можливостей	
обов'язкові		control-runtime, shape-validation, content-identity, local-serialization, query-projection	
additional		filesystem-inventory, export-scan, signature, build-dependency	
Можливість	Protocol ID	Operations	Receipt
control-runtime	promin.control-runtime.v1	continue, doctor, healthcheck, init, next, provider-binding, status, validate	content-addressed-receipt
shape-validation	promin.shape-validation.v1	command, continue, export, healthcheck, import, provider-binding, rebuild, replay	content-addressed-receipt
content-identity	promin.content-identity.v1	digest-bytes, digest-file, digest-value, healthcheck, provider-binding	content-addressed-receipt
local-serialization	promin.local-serialization.v1	atomic-replace, directory-sync, file-sync, healthcheck, local-writer-lock, provider-binding, recover	content-addressed-receipt
query-projection	promin.query-projection.v1	continue, healthcheck, next, provider-binding, rebuild, status	content-addressed-receipt

Можливість	Protocol ID	Operations	Receipt
filesystem-inventory	promin.filesystem-inventory.v1	healthcheck, immutable-tree-object, immutable-tree-stream, provider-binding	content-addressed-receipt
export-scan	promin.export-scan.v1	bounded-export-scan, healthcheck, provider-binding	content-addressed-receipt
signature	promin.signature.v1	healthcheck, provider-binding, sign-digest, verify-digest	content-addressed-receipt
build-dependency	promin.build-dependency.v1	healthcheck, import-build-facts, provider-binding	content-addressed-receipt

Кожна operation використовує Core-owned request/response shape та встановлений content-addressed dependency receipt. ProviderInvocationEvidence прив'язує точні protocol ID, operation і operation-contract digest, provider та adapter identity, invocation kind, dependency-receipt digest і observed outcome. Повний output прив'язує exact output_digest, output_size_bytes і output_size_ceiling_bytes; diagnostic stdout/stderr captures є окремими, обмежені 1 MiB і мають truncation flags. Unknown bindings, reconstructed receipts і raw provider-specific bypasses відхиляються.

Власник: core/semantic-model.json; presets/semantic-standard.json; core/policy-set.json; core/contracts.schema.json | Валідатор: technology_capabilities.adapter_protocol + ProviderInvocationEvidence + provider_preflight + provider_substitution

Робочі приклади

```
promin --root <project> init --standard-bundle <promin> --preset <preset> --project-plan
<plans/project.json> --standards-plan <plans/standards.json> --technologies-plan <plans/technologies.json>
--licenses-plan <plans/licenses.json> --authority-plan <plans/authority.json> --emit-plan <canonical-plans>
promin --root <project> init --standard-bundle <promin> --preset <preset> --project-plan
<plans/project.json> --standards-plan <plans/standards.json> --technologies-plan <plans/technologies.json>
--licenses-plan <plans/licenses.json> --authority-plan <plans/authority.json> --review-plan
promin --root <project> init --standard-bundle <promin> --preset <preset> --project-plan
<plans/project.json> --standards-plan <plans/standards.json> --technologies-plan <plans/technologies.json>
--licenses-plan <plans/licenses.json> --authority-plan <plans/authority.json> --dry-run
promin --root <project> init --standard-bundle <promin> --preset <preset> --project-plan
<plans/project.json> --standards-plan <plans/standards.json> --technologies-plan <plans/technologies.json>
--licenses-plan <plans/licenses.json> --authority-plan <plans/authority.json>
promin --root <project> init --standard-bundle <promin> --preset <preset> --project-plan
<plans/project.json> --standards-plan <plans/standards.json> --technologies-plan <plans/technologies.json>
--licenses-plan <plans/licenses.json> --authority-plan <plans/authority.json> --activation-proofs
<proofs.json>

promin --root <project> doctor
promin --root <project> status
promin --root <project> next --subject <id> --grant <holder-grant> --query-grant <query-grant> --depth 2
promin --root <project> validate
promin --root <project> continue <token> --subject <id> --grant <query-grant>
```

Приклад не обходить ingress: кожна дія зміни стану проходить canonical parse, schema, policies, перевірку цілісності Activation, capability/scope, Grant/SOD/Lease/fence, прив'язку evidence й atomic commit. `status`, `next` та `continue` відхиляють stale projection або continuation.

`doctor` є єдиним aggregate live health report для exact current components core, init, providers, recovery, replay і projection. Component і rollup statuses: healthy, degraded, failed або incomplete з precedence failed, incomplete, degraded, healthy. `--no-replay` дає incomplete; missing або stale projection ніколи не healthy; provider status походить із фактичних bounded observations. Report завжди має `report\_authoritative=false`, `pass\_credit=false`, `product\_acceptance\_pass=false` і `product\_public\_approval=not\_approved`.

OperationMetrics є live-only non-authoritative observation однієї команди й містить лише Core-owned duration, changed-record, event-write, projection-update, runtime-checkpoint, physical-payload-byte та bytes-per-changed-record fields. DoctorResult і OperationMetrics не збирають, не вигадують і не розподіляють token, cost, usage, goal, session, batch, workspace або per-run usage or per-run accounting.

Власник: presets/semantic-standard.json; core/policy-set.json; core/contracts.schema.json | Валідатор: base_user_commands + command_capability_scope + critical record definitions

Кандидат стандарту, докази й рішення

Ланцюг є ациклічним: точні bytes ZIP-кандидата утворюють StandardReleaseCandidateBinding; exact-package verification і multi-platform closure обчислюються повторно, а не приймаються як evidence roles; фізично перевірені результати утворюють StandardReleaseEvidenceManifest; після цього активне налаштоване повноваження підписує StandardReleaseDecision з явним outcome approve або reject; статус дистрибуції є похідним від перевіреного ланцюга. VERSION.json описує лише версію та ідентичність пакета. Кожний зовнішній JSON record парситься з одного bounded stable read тих самих bytes. Окремий рядок digest не доводить ані evidence, ані повноваження, а рішення не змінює bytes кандидата.

Поле StandardReleaseCandidateBinding	Прив'язка
record_type	точна ідентичність кандидата
standard_name	точна ідентичність кандидата
version	точна ідентичність кандидата
archive_sha256	точна ідентичність кандидата
archive_bytes	точна ідентичність кандидата
archive_member_manifest_digest	точна ідентичність кандидата
package_manifest_digest	точна ідентичність кандидата
checksums_digest	точна ідентичність кандидата
core_bundle_digest	точна ідентичність кандидата
preset_digest	точна ідентичність кандидата
package_tool_digest	точна ідентичність кандидата
validator_digest	точна ідентичність кандидата
test_manifest_digest	точна ідентичність кандидата
portable_implementation_closure_digest	точна ідентичність кандидата
candidate_binding_digest	точна ідентичність кандидата
Поле StandardReleaseEvidenceManifest	Прив'язка
record_type	фізично перевірений набір evidence
standard_name	фізично перевірений набір evidence
version	фізично перевірений набір evidence
candidate_binding_digest	фізично перевірений набір evidence
entries	фізично перевірений набір evidence
evidence_manifest_digest	фізично перевірений набір evidence

Обов'язкові полі evidence: linux, windows, physical-scale, saturation-audit, human-documents, linux-no-degradation, windows-no-degradation

Поле StandardReleaseDecision	Прив'язка
record_type	підписане твердження approve або reject
decision_id	підписане твердження approve або reject
standard_name	підписане твердження approve або reject
version	підписане твердження approve або reject
candidate_binding_digest	підписане твердження approve або reject
evidence_manifest_digest	підписане твердження approve або reject
outcome	підписане твердження approve або reject
decider_id	підписане твердження approve або reject
release_capability	підписане твердження approve або reject
trust_root_id	підписане твердження approve або reject
signature_provider_id	підписане твердження approve або reject
key_id	підписане твердження approve або reject
nonce	підписане твердження approve або reject
decided_at	підписане твердження approve або reject

Поле StandardReleaseDecision	Прив'язка
signed_claim_digest	підписане твердження approve або reject
signature	підписане твердження approve або reject

Перевірка Decision потребує release_capability=standard.distribute, налаштованих trust root і Ed25519 provider, активних відповідних key і decider, обмежених nonce та часу рішення, canonical signed-claim digest, чинного signature й окремо заданого SHA-256 pin точних bytes trust configuration. Чинний signature під неприкріпленим caller-supplied root має лише статус signature_valid_under_supplied_root і не може схвалити дистрибуцію. Невідомі, відкликані, прострочені, непідписані, повторно використані для іншої версії, unpinned або невідповідні записи відхиляються. Approve впливає лише на дистрибуцію стандарту; product acceptance залишається false, public approval - not approved, public_release_approved - false, а cutover і deletion залишаються false до виконання окремої межі людського рішення.

Перевірка документів

HumanDocumentVerification є типізованим результатом, прив'язаним до точного ZIP-кандидата. Він перевіряє всі чотири PDF за точними digest, strict parse, кількістю сторінок, кількістю вилучених символів, діагностикою порожніх сторінок і extraction, точними digest regular/bold/italic/mono fonts та deterministic rebuild. Кожна сторінка рендериться у digest-bound PNG, а visual_review_score охоплює всі rendered pages і результати clipping; перевірка лише першої сторінки або невизначена візуальна область не може пройти. Запис завжди зберігає product_acceptance_pass=false.

Поле HumanDocumentVerification	Структурне правило
candidate_binding_digest	\$ref="#/\$defs/Digest"
deterministic_rebuild	const=true
documents	type="array"; minItems=4; maxItems=4; uniqueItems=true
extraction_diagnostics	type="object"; additionalProperties=false
font_bindings	type="object"; additionalProperties=false
generator_result_digest	\$ref="#/\$defs/Digest"
invocation	\$ref="#/\$defs/ReleaseEvidenceInvocation"
page_count	type="integer"; minimum=1
producer	\$ref="#/\$defs/ReleaseEvidenceProducer"
producer_attestation	\$ref="#/\$defs/EvidenceProducerAttestation"
product_acceptance_pass	const=false
raw_artifact_manifest_digest	\$ref="#/\$defs/Digest"
rebuild_digest	\$ref="#/\$defs/Digest"
record_type	const="HumanDocumentVerification"
render_environment	type="object"; additionalProperties=false
render_manifest	type="array"; minItems=4; maxItems=4096; uniqueItems=true
render_manifest_digest	\$ref="#/\$defs/Digest"
result_digest	\$ref="#/\$defs/Digest"
status	const="pass"
visual_review_scope	type="object"; additionalProperties=false

- P-045 current-release-closure: A project-operational release Decision is immutable history; current product release eligibility is derived and becomes false after a blocker, failed gate, stale or missing evidence, implementation drift, or Activation drift. It is separate from the external StandardReleaseDecision for promin distribution.
- P-054 cross-platform-release-evidence: The exact candidate ZIP must pass authenticated Windows and Linux platform observations, clean-install, package, handle-relative path/race, locking, crash, replay, physical scale, offline installed no-degradation, online compatibility, dependency/SBOM/RECORD/license closure, and typed PDF gates. One StandardReleaseEvidenceManifest physically resolves exactly seven unique authority roles, the

exact non-authoritative eight-lane CPython 3.13/3.14 online/offline matrix, all four CPython 3.14supplemental records, and every nested physical record by path, SHA-256, and byte count. Allproducer attestations bind the same candidate and participate in one replay/SOD/chronology graph;matrix_authoritative, pass_credit, acceptance_pass, product_acceptance_pass, and public approvalremain false.

- P-055 standard-release-separation: An external StandardReleaseDecision has explicit approve or reject outcome, binds candidate and evidence manifest digests, follows the manifest-derived maximum evidence completion time, stays within 300 seconds of verifier clock skew, and verifies decider, standard.distribute capability, independently pinned trust root, Ed25519 provider, event-time key validity, nonce, signed claim digest, and signature. The signed decision remains immutable history; current eligibility is derived; product_acceptance_pass is false and product_public_approval is not_approved.
- P-056 required-gates-fail-closed: No-degradation runs the complete required suite only through the clean installed interpreter outside the install source, with isolated mode, cleared PYTHONPATH, import-origin assertion, zero skipped tests, and full hash-bound JUnit/stdout/stderr bytes. Distribution acceptance executes every required positive, mutation, concurrency, crash, replay, path, package, platform, Python-minor, offline/online, scale, search, PDF, evidence-resolution, chronology, and signature gate; a skipped, missing, stale, blocked, failed, unsigned, unresolved, source-shadowed, or unpinned gate receives no pass credit.

Власник: core/authority-model.json; core/contracts.schema.json; core/conformance.json | Валідатор: candidate binding + physical evidence resolution + configured signed approve/reject + typed PDF verification